Git para economistas

Introducción a sistemas de control de versión

Nicolas Franz-Pattillo

October 20, 2020

University of British Columbia

Table of contents

1. Introducción

2. Git

    Qué es Git?

    Cómo se usa Git?

    Trabajo no lineal

3. Interfaces de usuario, “la nube” y redes sociales

4. Buenas prácticas y Conclusiones

Introducción

1. Introducción

2. Git

3. Interfaces de usuario, “la nube” y redes sociales

4. Buenas prácticas y Conclusiones

Introducción

- Hoy en día no es raro que los proyectos contengan mas de un archivo.
  - Texto
  - Datos y gráficos
  - Programas: *.do, *.m, *.prg, etc.

Introducción

- Hoy en día no es raro que los proyectos contengan mas de un archivo.
  - Texto
  - Datos y gráficos
  - Programas: *.do, *.m, *.prg, etc.
- Tampoco es raro que tengan varios co-autores.

Introducción

- Hoy en día no es raro que los proyectos contengan mas de un archivo.
  - Texto
  - Datos y gráficos
  - Programas: *.do, *.m, *.prg, etc.
- Tampoco es raro que tengan varios co-autores.

Problema

Cómo hacemos avanzar el proyecto de forma ordenada?

Introducción

- Hoy en día no es raro que los proyectos contengan mas de un archivo.
  - Texto
  - Datos y gráficos
  - Programas: *.do, *.m, *.prg, etc.
- Tampoco es raro que tengan varios co-autores.

Problema

Cómo hacemos avanzar el proyecto de forma ordenada?

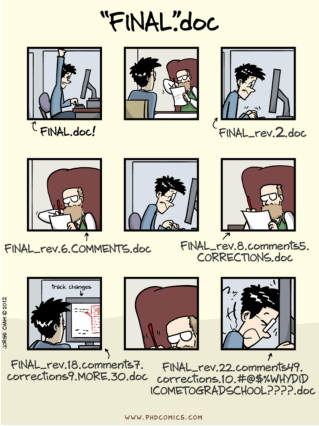
$\Longrightarrow$

Solución*

Coordinando una forma de hacer los cambios

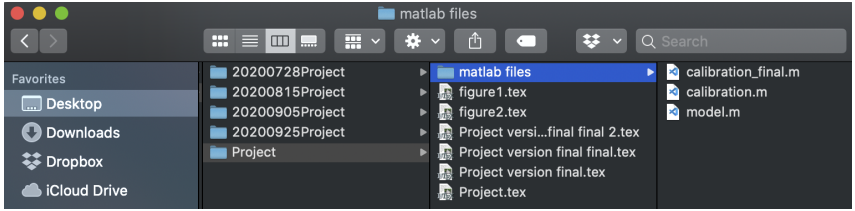
Qué es un sistema de control de versión?

Figure 1: PhD Comics # 1531



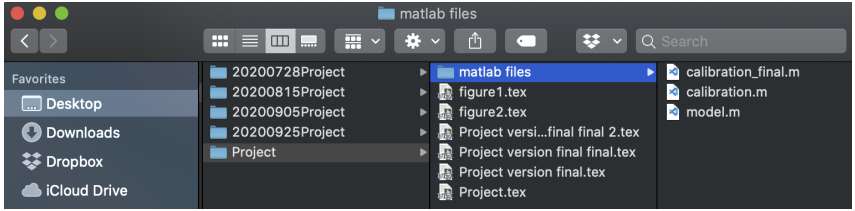
Qué es un sistema de control de versión?

Figure 2: Control de versión “naive”



Qué es un sistema de control de versión?

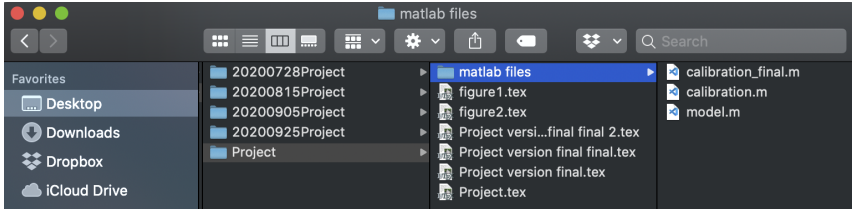
Figure 2: Control de versión “naive”



- Los programadores han tenido este problema por décadas.

Qué es un sistema de control de versión?

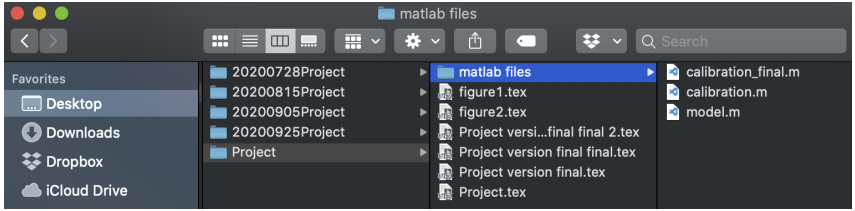
Figure 2: Control de versión “naive”



- Los programadores han tenido este problema por décadas.
- Las soluciones (*software*) han ido evolucionando con el tiempo. En general se les conoce como **sistemas de control de versión**

Qué es un sistema de control de versión?

Figure 2: Control de versión “naive”



- Los programadores han tenido este problema por décadas.
- Las soluciones (*software*) han ido evolucionando con el tiempo. En general se les conoce como **sistemas de control de versión**
- La mayoría de ellos presenta al usuario la versión que se quiere revisar y deja el resto fuera de la vista.

Repo, commit, working copy ... me no english!

Los *softwares* de control de versión comparten una terminología común (al igual que muchos lenguajes de programación comparten el condicional *if... then... else* y los bucles *for* o *while*)

1. **Commit**: Fotografía del proyecto en un momento del tiempo determinado

Repo, commit, working copy ... me no english!

Los *softwares* de control de versión comparten una terminología común (al igual que muchos lenguajes de programación comparten el condicional *if... then... else* y los bucles *for* o *while*)

1. **Commit**: Fotografía del proyecto en un momento del tiempo determinado

2. **Repository**: ("Repo") Proyecto completo, incluyendo su historia.

Repo, commit, working copy ... me no english!

Los *softwares* de control de versión comparten una terminología común (al igual que muchos lenguajes de programación comparten el condicional *if... then... else* y los bucles *for* o *while*)

1. **Commit**: Fotografía del proyecto en un momento del tiempo determinado

2. **Repository**: ("Repo") Proyecto completo, incluyendo su historia.

3. **Working copy**: Versión del proyecto en la cual se está trabajando.

Tipos de control de versión

1. Locales
2. Distribuidos
3. No distribuidos

Tipos de control de versión

Figure 3: Sistema Distribuido

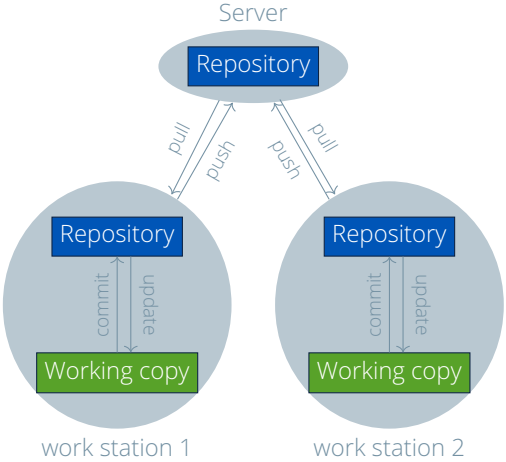
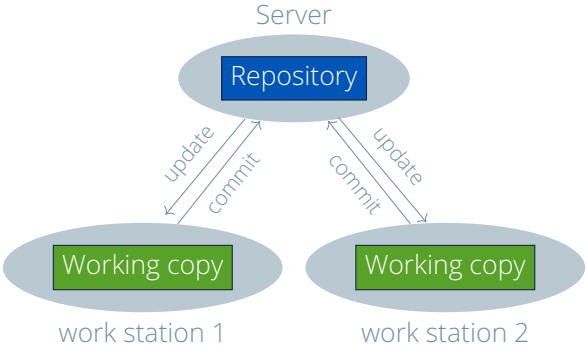


Figure 4: Sistema No Distribuido



Preguntas?

Git

1. Introducción

2. Git

3. Interfaces de usuario, “la nube” y redes sociales

4. Buenas prácticas y Conclusiones

Qué es Git?

- Git es un sistema distribuido de control de versión. El más utilizado en el mundo.

Qué es Git?

- Git es un sistema distribuido de control de versión. El más utilizado en el mundo.

- El *software* de Git se instala “localmente”. Para empezar a utilizar Git no es necesario un servidor. No obstante, este último mejora la colaboración significativamente.

Qué es Git?

- Git es un sistema distribuido de control de versión. El más utilizado en el mundo.

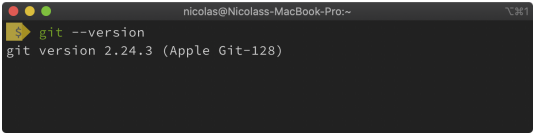
- El *software* de Git se instala "localmente". Para empezar a utilizar Git no es necesario un servidor. No obstante, este último mejora la colaboración significativamente.

- Git no es lo mismo que GitHub! GitHub es una red social que utiliza la arquitectura Git como base. Otros ejemplos son GitLab y BitBucket.

Cómo se usa Git?

- Git se usa a través de la linea de comando.
  - MacOS: Terminal
  - Windows: Command Prompt o PowerShell

Figure 5: iTerm terminal en MacOs

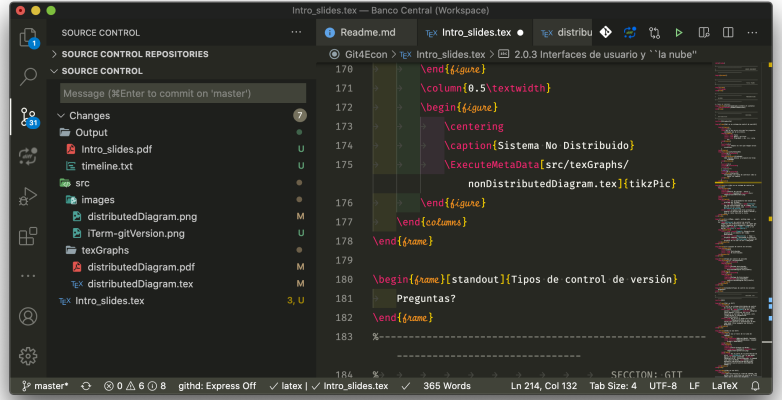


- Git es parte de la distribución de otros *softwares*, por lo que puede que ya lo tengas instalado. (e.g., Python-Anaconda, o command line tools de MacOs, Matlab 2020b).

Cómo se usa Git?

- Hay muchas otras interfaces de usuario gráficas (GUI) que lo hacen mas amigable

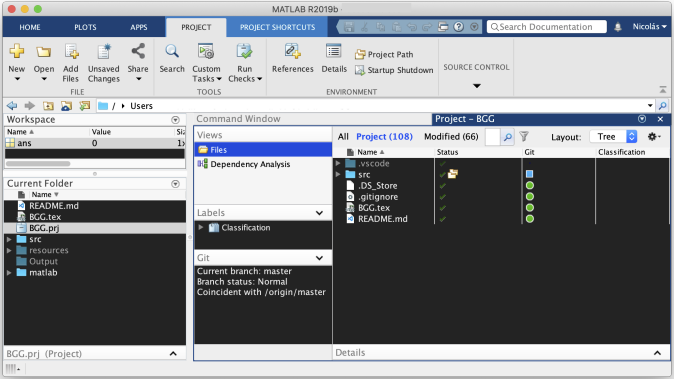
Figure 6: Integración Git, VS Code



Cómo se usa Git?

- Hay muchas otras interfaces de usuario gráficas (GUI) que lo hacen mas amigable

Figure 7: Integración Git, Matlab



Cómo se usa Git?

- La forma estándar de aprender Git es a través de la línea de comando.

Cómo se usa Git?

- La forma estándar de aprender Git es a través de la línea de comando.

- Esto no quiere decir que el usuario lo use a través de ella, pero le da uniformidad al proceso (Las GUI pueden tener formas diversas de implementar Git y aprenderlas es difícil si no se entiende el objetivo. Es como querer aprender MS excel sin saber matemáticas)

Cómo se usa Git?

- La forma estándar de aprender Git es a través de la línea de comando.

- Esto no quiere decir que el usuario lo use a través de ella, pero le da uniformidad al proceso (Las GUI pueden tener formas diversas de implementar Git y aprenderlas es difícil si no se entiende el objetivo. Es como querer aprender MS excel sin saber matemáticas)

- Con el tiempo, el uso y la terminología Git se vuelve natural...

Cómo se usa Git?

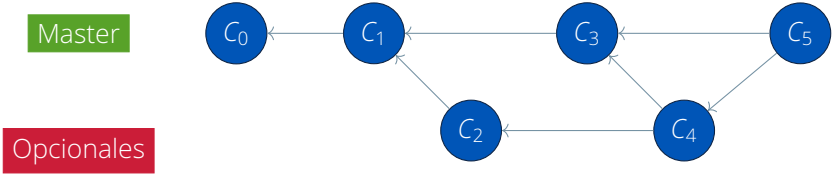
- La forma estándar de aprender Git es a través de la línea de comando.
- Esto no quiere decir que el usuario lo use a través de ella, pero le da uniformidad al proceso (Las GUI pueden tener formas diversas de implementar Git y aprenderlas es difícil si no se entiende el objetivo. Es como querer aprender MS excel sin saber matemáticas)
- Con el tiempo, el uso y la terminología Git se vuelve natural... trust me!

Preguntas?

Trabajo no lineal

- Una de las grandes ventajas de Git es que simplifica el uso de “ramas” o estructuras no lineales

Figure 8: Árbol Git, ejemplo



Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.
3. Hacemos el primer *commit* con ambos archivos para luego empezar a escribir un documento en LaTeX

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.
3. Hacemos el primer *commit* con ambos archivos para luego empezar a escribir un documento en LaTeX
4. Hacemos un segundo *commit* incluyendo el texto anterior.

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.
3. Hacemos el primer *commit* con ambos archivos para luego empezar a escribir un documento en LaTeX
4. Hacemos un segundo *commit* incluyendo el texto anterior.
5. Creamos la rama “Opcionales” para explorar otros parámetros.

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.
3. Hacemos el primer *commit* con ambos archivos para luego empezar a escribir un documento en LaTeX
4. Hacemos un segundo *commit* incluyendo el texto anterior.
5. Creamos la rama “Opcionales” para explorar otros parámetros.
6. Hay un error en la solución del modelo! Volvemos a la rama “master” para solucionarlo.

Trabajo no lineal

Ejemplo A: el uso de ramas para experimentar

1. Empezamos a trabajar en un nuevo modelo estructural.
2. Inicializamos el repositorio git y creamos algunos códigos: solución y parametros base.
3. Hacemos el primer *commit* con ambos archivos para luego empezar a escribir un documento en LaTeX
4. Hacemos un segundo *commit* incluyendo el texto anterior.
5. Creamos la rama “Opcionales” para explorar otros parámetros.
6. Hay un error en la solución del modelo! Volvemos a la rama “master” para solucionarlo.
7. Volvemos a explorar en “Opcionales” (actualizando la solución) y llevamos los resultados de “Opcionales” a “Master”

Ejemplo A

Cómo inicializar un repositorio

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject                ⌥⌘1
Last login: Tue Oct 20 13:45:52 on ttys000

~                                                   ✔  1453  18:04:24
$ cd Desktop/newProject

~/Desktop/newProject                                ✔  1454  18:04:43
$ git init
Initialized empty Git repository in /Users/nicolas/Desktop/newProject/.git/

~/Desktop/newProject  master                        ✔  1455  18:04:58
$
```

Ejemplo A

Una vez que tenemos cambios que queramos comprometer, los podemos organizar en el *staging area* (separar cambios en distintos *commits*, por ejemplo)

Staging

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject
$

~/Desktop/newProject  master ?                    ✔  1455  18:32:54
$ git add -A

~/Desktop/newProject  master +                    ✔  1456  18:33:14
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   main.m
        new file:   solution.m
```

Ejemplo A

Una vez que ejecutamos el *commit*, solos los cambios en el *staging area* quedarán registrados.

Commit

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject                    ⌥⌘1
$ git commit -m "Initial commit"
[master (root-commit) e2f1c62] Initial commit
 2 files changed, 13 insertions(+)
 create mode 100644 main.m
 create mode 100644 solution.m

~/Desktop/newProject  master                          ✔  1458  18:34:34
$
```

Ejemplo A

Segundo commit y git log

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject
$ git commit -m "report added"
[master 4be3dd5] report added
 1 file changed, 4 insertions(+)
 create mode 100644 report.tex

~/Desktop/newProject  master ?                    ✔  1462  18:59:59
$ git log --all --decorate --oneline --graph
```

```
git log --all --decorate --oneline --graph
* 4be3dd5 (HEAD -> master) report added
* e2f1c62 Initial commit
(END)
```

Ejemplo A

Nueva rama: git branch + git checkout

Ahora creamos la rama “Opcionales” en donde cambiaremos los parametros

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject
$ git checkout -b opcionales
Switched to a new branch 'opcionales'
```

Una vez que hacemos los cambio en el archivo “main.m” hacemos el *staging* y *commit*

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject
~/Desktop/newProject  opcionales ?                    ✔  1470  19:14:32
$ git add main.m

~/Desktop/newProject  opcionales + ?                  ✔  1471  19:16:50
$ git commit -m "change in parameters #1"
[opcionales 02df333] change in parameters #1
 1 file changed, 1 insertion(+), 1 deletion(-)

~/Desktop/newProject  opcionales ?                    ✔  1472  19:17:48
```

Ejemplo A

Git diff

Para visualizar los cambios que hicimos existe el comando *git diff*, seguido por identificadores de los *commits* a comparar y opciones del comando

```
~/Desktop/newProject  opcionales ?                    ✓  1480  20:23:21
$  git diff 02df333 4be3dd5 --color-words
```

```
git diff 02df333 4be3dd5 --color-words                    ⌥⌘1
diff --git a/main.m b/main.m
index 8055a76..ee86fe1 100644
--- a/main.m
+++ b/main.m
@@ -2,7 +2,7 @@
clc

% Parameters
p = [0.11, 20.5, 0.050.01];
% Solution
sol = solution(p);
% Plot
(END)
```

Ejemplo A

Git checkout

Ahora nos acabamos de dar cuenta de un error en *solucion.m*. Dado que el error involucra a todo el repositorio, es mejor resolver el problema en “master”. Para ello tenemos que saltar de rama

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject                    ⌥⌘1
$ git checkout master
Switched to branch 'master'

~/Desktop/newProject  master ?                    ✔  1481  20:33:45
$ _
```

Ejemplo A

Git checkout

Ahora nos acabamos de dar cuenta de un error en *solucion.m*. Dado que el error involucra a todo el repositorio, es mejor resolver el problema en “master”. Para ello tenemos que saltar de rama

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject                    ⌥⌘1
$ git checkout master
Switched to branch 'master'

~/Desktop/newProject  master ?                          ✔  1481  20:33:45
$ _
```

Una vez resuelto el problema, y comprometidos los cambios, podemos seguir trabajando en nuestra rama opcionales

Ejemplo A

Git checkout

Ahora nos acabamos de dar cuenta de un error en *solucion.m*. Dado que el error involucra a todo el repositorio, es mejor resolver el problema en “master”. Para ello tenemos que saltar de rama

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject                    ⌥⌘1
$ git checkout master
Switched to branch 'master'

~/Desktop/newProject  master ?                        ✔  1481  20:33:45
$ _
```

Una vez resuelto el problema, y comprometidos los cambios, podemos seguir trabajando en nuestra rama opcionales ... excepto por...

Ejemplo A

git merge

El arreglo en “master” no esta disponible en ”opcionales”. Para llevar la linea de “master” a “opcionales” nos cambiamos a la segunda y hacemos un *merge* con la primera.

```
nicolas@Nicolass-MacBook-Pro:~/Desktop/newProject
$

~/Desktop/newProject  master ?                                  ✓ 1518 20:50:01
$ git checkout opcionales
Switched to branch 'opcionales'

~/Desktop/newProject  opcionales ?                              ✓ 1519 20:50:11
$ git merge master
Merge made by the 'recursive' strategy.
 solution.m | 1 +
 1 file changed, 1 insertion(+)

~/Desktop/newProject  opcionales ?                              ✓ 1520 20:51:50
$
```

Ejemplo A

Visualización del arbol git

```
git log --all --decorate --oneline --graph
*   656e502 (HEAD -> opcionales) Merge branch 'master' into opcionales
|\
| * 15419bb (master) solution fixed
* | 02df333 change in parameters #1
|/
* 4be3dd5 report added
* e2f1c62 Initial commit
(END)
```

Ahora podemos evaluar nuestras pruebas en “opcionales”. Si los cambios nos gustan, los podemos incluir en nuestra rama “master” haciendo un *merge*, esta vez de “opcionales” a “master”.

Ejemplo A

Merge y branch -d

Una vez que ejecutamos el *merge*, eliminar la rama “opcionales” ($ *git branch -d opcionales*) no elimina la información de los commits hechos en ella

```
git log --all --decorate --oneline --graph
* 27f4e52 (HEAD -> master) running the model with 2 sets of parameters
*   656e502 Merge branch 'master' into opcionales
|\
| * 15419bb solution fixed
* | 02df333 change in parameters #1
|/
* 4be3dd5 report added
* e2f1c62 Initial commit
(END)
```

1. Introducción

2. Git

3. Interfaces de usuario, “la nube” y redes sociales

4. Buenas prácticas y Conclusiones

Buenas prácticas y Conclusiones

1. Introducción

2. Git

3. Interfaces de usuario, “la nube” y redes sociales

4. Buenas prácticas y Conclusiones